

Tecnología Digital V: Diseño de algoritmos

TP2 - Logística centralizada de primera milla

Autores:

Francisca Goldstein, Victoria Iasillo e Isaac Silva



1. Introducción al problema

ThunderPack es una start-up que cuenta con una red de depósitos que reciben los paquetes de distintos vendedores con el fin de entregarlos a un camión recolector que los llevará a sus respectivos destinos. Actualmente, los vendedores eligen de forma independiente en qué depósitos dejar sus paquetes, lo que puede generar una utilización ineficiente de la capacidad disponible de los depósitos y a su vez aumentar las distancias recorridas por los vendedores para llegar a dichos depósitos. Para este trabajo práctico, se propone una solución al problema modelándolo como un Problema de Asignación Generalizada (GAP), donde el objetivo es asignar cada vendedor a un único depósito respetando las restricciones de capacidad de los depósitos y minimizando la distancia total recorrida por los vendedores. Para esto se implementan y comparan distintas estrategias de resolución basadas en heurísticas constructivas, métodos de búsqueda local y metaheurísticas, evaluando su desempeño en distintos aspectos.

2. Clases utilizadas

Se definen solo dos clases para este trabajo:

2.1. GAPInstance

Tiene los datos ordenados del problema que queremos resolver. Con el método `readFromFile`, lee el archivo y crea atributos como:

- `M`: cantidad de depósitos
- `N`: cantidad de vendedores
- `vector<vector<float>> costo: costo[i][j]`: costo de asignar el vendedor `j` al depósito `i`
- `vector<vector<float>> demanda: demanda[i][j]`: recurso que consume el vendedor `j` en el depósito `i`
- `vector<float> capacidad: capacidad[i]`: capacidad total del depósito `i`

Y además la clase cuenta con otros métodos que devuelven la demanda de un vendedor `j` para un depósito `i` (`getdemanda(i, j)`), el costo de un vendedor a un depósito (`getcosto(i, j)`), la capacidad de un depósito (`getcapacidad(i)`) y el costo máximo entre todos los vendedores (`getcosto_maximo`).

Fue muy útil haber creado esta clase por la cantidad de veces que se usan tanto los atributos como los métodos para trabajar con las distintas funciones del programa.

2.2. GAPSolution

Una vez que se procesa una instancia y se realiza una asignación se crea un objeto de la clase `GAPSolution` cuyos atributos son:

- `vector<int> asignacin`: un vector de números con los índices de los vendedores asignados a cada depósito.
- `vector<float> capacidad_residual`: la capacidad restante de cada depósito luego de haberles asignado los vendedores en esa solución.
- `float costo_total`: el costo total resultante de la solución.

El objeto de esta clase siempre se construye en base a una instancia (objeto de la clase **GAPInstance**), eso hace que la solución inicial siempre sea que ningún vendedor está asignado a un depósito. Esto se hace definiendo el costo total igual cero, ya que luego las heurísticas propuestas se encargan de manejar el costo de cada vendedor según corresponda.

Además, se crearon métodos como:

- **es_factible_asignar**: indica si es posible asignar un vendedor a un depósito dentro de esa solución.
- **asignar**: asigna un vendedor a un depósito dentro de esa solución. Dado un vendedor y un depósito (pudiendo este último ser **-1** si el vendedor queda sin asignar), la función actualiza el vector **asignacion** (vector de tamaño **n**) registrando en la posición del vendedor el depósito elegido. Si el depósito no es el depósito ficticio **-1**, descuenta la demanda del vendedor de la capacidad residual de ese depósito y suma el costo correspondiente al costo total de la solución. Cuando el depósito es **-1**, no se modifica capacidad en este paso, sino que se le suma al costo total el costo máximo de entre todos los vendedores multiplicado por 3.
- **relocate**: se encarga de trasladar un vendedor a un nuevo depósito, siempre que el movimiento sea válido y factible. Primero verifica que el vendedor no esté ya asignado al depósito destino y que la nueva asignación respete las restricciones de capacidad. Luego guarda el depósito original del vendedor y actualiza su asignación al nuevo depósito. Si el vendedor estaba previamente asignado a algún depósito, se actualizan las capacidades residuales y el costo total de la solución, se libera en el depósito anterior la capacidad que ocupaba el vendedor y se descuenta del costo total el costo asociado a esa asignación. Además, si el depósito destino es válido, se descuenta de su capacidad residual la demanda del vendedor y se suma al costo total el costo correspondiente a la nueva asignación. De esta manera, la solución mantiene actualizadas tanto las capacidades disponibles como su costo total luego de realizar el movimiento.
- **swap**: intercambia dos vendedores de depósitos distintos y recalcula capacidades y costo total. Primero verifica si un intercambio de esas asignaciones de esos vendedores es factible. Luego, si pasó la verificación, liberamos los recursos consumidos por la asignación vieja, efectuamos el intercambio cruzado en el vector **asignacion** e impactamos el consumo de recursos y costos de la nueva asignación en los depósitos.
- **escribir_solucion**: escribe la solución en el formato pedido por los profesores en un archivo de nombre recibido por parámetro. Genera un archivo **.txt** con una línea por depósito, donde la línea **i**-ésima corresponde al depósito **i**; y en cada línea una lista de números, separados por un espacio, con los índices de los vendedores asignados a ese depósito **i**.

Algo interesante a destacar es que la clase no cuenta con los atributos en el **private** ya que necesitamos que sean públicos para permitir que los algoritmos de búsqueda local funcionen y los editen.

3. Algoritmos utilizados

3.1. Heurísticas

3.1.1. Greedy por vendedor

Recorremos el vector de vendedores en orden, y para cada uno evaluamos su compatibilidad con todos los depósitos: para cada depósito nos fijamos si la demanda del vendedor es compatible con la capacidad disponible, y en caso de que lo sea, guardamos su costo. A medida que recorremos los depósitos factibles, vamos comparando el costo actual con el mejor costo guardado hasta el momento, quedándonos siempre con el depósito de menor costo encontrado. Una vez recorridos todos los depósitos, si encontramos al

menos uno factible, le aplicamos la función `asignar()` al vendedor con el mejor depósito encontrado. En caso de que ningún depósito resulte factible para ese vendedor, también llamamos a `asignar()`, pero pasándole como depósito el valor -1 (depósito ficticio), que se encarga de registrar que el vendedor quedó sin asignar.

Algorithm 1 Greedy por vendedor

1:	for cada vendedor $j = 1 \dots n$ do	$O(n)$
2:	$mejor_costo \leftarrow \infty$	$O(1)$
3:	$mejor_depósito \leftarrow \text{ficticio}$	$O(1)$
4:	for cada depósito $i = 1 \dots m$ do	$O(m)$
5:	if <code>es_factible(j, i)</code> then	$O(1)$
6:	if $c[i][j] < mejor_costo$ then	$O(1)$
7:	$mejor_costo \leftarrow c[i][j]$	$O(1)$
8:	$mejor_depósito \leftarrow i$	$O(1)$
9:	end if	
10:	end if	
11:	end for	
12:	<code>asignar(j, mejor_depósito)</code>	$O(1)$
13:	end for	

La complejidad total es $O(n \cdot m)$.

3.1.2. Greedy por depósito con ajuste por flexibilidad

A diferencia de la heurística anterior, en esta recorreremos por depósito. De todas formas, comenzamos recorriendo los vendedores y guardando en un vector auxiliar `cantidad_depositos_factibles` la cantidad de depósitos factibles para cada uno, calculada con un contador. En caso de que este valor sea 0, se reemplaza por 1 para evitar divisiones por cero. Luego recorreremos los depósitos y, para cada uno, evaluamos todos los vendedores no asignados y compatibles con su capacidad. Para cada vendedor calculamos un puntaje definido como:

$$puntaje = \frac{costo}{costo_{max}} - 0,2 \times \frac{1}{(1 + cantidad_depositos_factibles)}$$

Donde el costo se normaliza dividiéndolo por el costo máximo de la instancia para llevarlo al rango $[0, 1]$ y hacerlo comparable con el término de flexibilidad, que busca favorecer a los vendedores con menos alternativas de asignación. A su vez, el factor 0,2 se introdujo para mantener el puntaje en una escala comparable a la del costo normalizado y evitar que la corrección por flexibilidad dominara completamente la decisión. A partir de este puntaje, se selecciona el vendedor con menor puntaje y se lo asigna al depósito. Si al finalizar quedan vendedores sin asignar, se los envía al depósito ficticio aplicando la penalización correspondiente.

Algorithm 2 Greedy por depósito con ajuste por flexibilidad

1: for cada vendedor $j = 1 \dots n$ do	$O(n)$
2: $f[j] \leftarrow \text{máx}(1, \text{cantidad de depósitos } i \text{ factibles para } j)$	$O(1)$
3: end for	
4: for cada depósito $i = 1 \dots m$ do	$O(m)$
5: repeat	
6: $\text{mejor_puntaje} \leftarrow \infty$	$O(1)$
7: $\text{mejor_vendedor} \leftarrow \text{ninguno}$	$O(1)$
8: for cada vendedor j no asignado do	$O(n)$
9: if $\text{es_factible}(j, i)$ then	$O(1)$
10: $\text{puntaje} \leftarrow (c[i][j]/\text{costo_máximo}) - 0,2 \times (1/(1 + f[j]))$	$O(1)$
11: if $\text{puntaje} < \text{mejor_puntaje}$ then	$O(1)$
12: $\text{mejor_puntaje} \leftarrow \text{puntaje}$	$O(1)$
13: $\text{mejor_vendedor} \leftarrow j$	$O(1)$
14: end if	
15: end if	
16: end for	
17: if $\text{mejor_vendedor} = \text{ninguno}$ then	
18: salir del repetir	$O(1)$
19: end if	
20: $\text{asignar}(\text{mejor_vendedor}, i)$	$\triangleright$ actualiza costo y capacidad $O(1)$
21: until $\text{mejor_vendedor} = \text{ninguno}$	
22: end for	
23: for cada vendedor j no asignado do	$O(n)$
24: $\text{asignar}(j, -1)$	$\triangleright$ aplica directamente la penalización $3c_{\max}$ $O(1)$
25: end for	

La complejidad total es $O(m \cdot n)$.

3.1.3. Greedy randomized

Esta función es parecida al greedy por vendedor, pero en lugar de elegir siempre el depósito factible de menor costo introduce una componente aleatorio. Para cada vendedor se recorren todos los depósitos y se arma una lista de candidatos con aquellos depósitos que son factibles junto con su costo de asignación. Si no se encuentra ningún depósito factible, se le asigna el valor -1 con la función **asignar**, correspondiente al depósito ficticio. En caso contrario, se ordenan los candidatos de menor a mayor costo, se toman los k mejores y se elige uno de ellos de forma aleatoria. Finalmente, se asigna el vendedor al depósito elegido mediante la función **asignar**. De esta manera, se sigue favoreciendo a los depósitos de menor costo, pero se generan soluciones distintas al no elegir siempre la mejor opción.

Algorithm 3 GreedyRandomizado(instancia, k)

1: Crear solución vacía	$O(1)$
2: for cada vendedor j do	$O(n)$
3: Crear lista de candidatos	
4: for cada depósito i do	$O(m)$
5: if i es factible para j then	
6: Agregar $(\text{costo}(i, j), i)$ a candidatos	$O(1)$
7: end if	
8: end for	
9: if candidatos está vacío then	
10: Asignar j al depósito ficticio (-1)	$O(1)$
11: else	
12: Ordenar candidatos por costo creciente	$O(n \log n)$
13: $\text{mínimo} \leftarrow \text{mín}(k, \text{cantidad de candidatos})$	$O(1)$
14: Tomar los primeros mínimo candidatos	$O(1)$
15: Elegir uno al azar	$O(1)$
16: Asignar j al depósito elegido	$O(1)$
17: end if	
18: end for	
19: Devolver solución	$O(1)$

La complejidad total es $O(n^2 \cdot m \cdot \log n)$.

3.2. Búsqueda local

3.2.1. Relocate (First improvement)

Se parte de una solución inicial obtenida mediante una heurística y se recorren todos los vendedores utilizando el índice j . Para cada vendedor, se identifica el depósito i al que está actualmente asignado y se evalúan todos los demás depósitos k . En cada caso, se verifica que k sea distinto de i y que la demanda del vendedor en el depósito destino no supere la capacidad residual disponible. Además, se compara el costo de asignar el vendedor al depósito k con el costo de mantenerlo en el depósito actual i . Si el movimiento es factible y produce una reducción en el costo, se ejecuta la operación **relocate**(j , k). Como se utiliza una estrategia de First Improvement, la búsqueda se detiene al encontrar la primera mejora y vuelve a comenzar el recorrido desde el inicio. Este proceso se repite hasta que no se encuentran movimientos que permitan seguir mejorando la solución.

Algorithm 4 Relocate (First Improvement)

1: $\text{mejoro} \leftarrow \text{verdadero}$	$O(1)$
2: while mejoro do	
3: $\text{mejoro} \leftarrow \text{falso}$	$O(1)$
4: for cada vendedor j do	$O(n)$
5: $i \leftarrow$ depósito asignado a j	$O(1)$
6: for cada depósito k do	$O(m)$
7: if $k \neq i$ y $\text{demanda}(k, j) \leq \text{capacidad_residual}(k)$ y $\text{costo}(k, j) < \text{costo}(i, j)$ then	$O(1)$
8: relocate (j, k)	$O(1)$
9: $\text{mejoro} \leftarrow \text{verdadero}$	$O(1)$
10: end if	
11: end for	
12: end while	
13: end while	

La complejidad total es $O(n \cdot m)$.

3.2.2. Swap (First improvement)

El algoritmo se encarga de intercambiar asignaciones de vendedores a depósitos llamando al método de la clase **GAPSolution**, **swap**. El funcionamiento es el siguiente: para cada vendedor se decide si un intercambio con cada uno de los otros vendedores sería factible y beneficioso porque llevaría a un menor costo total. Es decir, se chequea que si se asigna un vendedor j al depósito asignado a un vendedor h y viceversa, las capacidades de ambos depósitos no serían superadas por las demandas de los nuevos vendedores asignados. Por supuesto que esto viene acompañado de verificar que, si se realiza ese reemplazo, el costo total resultante debe ser menor al que se tenía antes de hacer ese intercambio.

Algorithm 5 Swap (First Improvement)

1:	<i>mejoro</i> $\leftarrow$ verdadero	$O(1)$
2:	while <i>mejoro</i> do	
3:	<i>mejoro</i> $\leftarrow$ falso	$O(1)$
4:	for cada vendedor j (mientras <i>mejoro</i> = falso) do	$O(n)$
5:	<i>i</i> $\leftarrow$ depósito asignado a j	$O(1)$
6:	for cada vendedor h do	$O(n)$
7:	<i>q</i> $\leftarrow$ depósito asignado a h	$O(1)$
8:	if $\text{demanda}(i, h) \leq \text{capacidad_residual}(i) + \text{demanda}(i, j)$ y $\text{demanda}(q, j) \leq \text{capacidad_residual}(q) + \text{demanda}(q, h)$ y $\text{costo}(q, j) + \text{costo}(i, h) < \text{costo}(i, j) + \text{costo}(q, h)$ then	$O(1)$ $\triangleright$ se chequea que el swap sea factible según capacidades y demandas, y que produzca menor costo
9:	swap (j, h)	$O(1)$
10:	<i>mejoro</i> $\leftarrow$ verdadero	$O(1)$
11:	end if	
12:	end for	
13:	end for	
14:	end while	

La complejidad total es $O(n^2)$.

3.3. Metaheurística

Nuestra propuesta de metaheurística fue la de GRASP. El proceso consta de pasar una instancia del problema por un algoritmo goloso randomizado (**greedy_randomizado**), luego por uno de búsqueda local de tipo **Swap** y por último por uno de **Relocate**. Repetimos este proceso un número de veces igual al mínimo entre 500 y 10 veces la cantidad de vendedores. Esto lo hacemos para encontrar una cantidad óptima y razonable de iteraciones. Cuando terminamos, nos habremos guardado la solución con menor costo entre todas las que conseguimos durante esas iteraciones.

Algorithm 6 GAPSolution grasp(const GAPInstance& inst, int k)

1: $res \leftarrow$ GAPSolution(inst)	$O(n + m)$
2: $max \leftarrow \min(500, 10 \cdot \#vendedores)$	$O(1)$
3: $mejor \leftarrow \infty$	$O(1)$
4: for $i = 1 \dots max$ do	$O(n)$
5: $s \leftarrow$ greedy_randomizado(inst, k)	$O(n^2 \cdot m \cdot \log n)$
6: swap_busqueda_local(s , inst)	$O(n^2)$
7: relocate_busqueda_local(s , inst)	$O(n \cdot m)$
8: if $s.costo_total < mejor$ then	$O(1)$
9: $mejor \leftarrow s.costo_total$	$O(1)$
10: $res \leftarrow s$	$O(1)$
11: end if	
12: end for	
13: Devolver res	$O(1)$

Complejidad: $O(n^3 \cdot m \cdot \log n)$. Aclaramos que la complejidad de crear un objeto de tipo GAPSolution es de $O(n + m)$ por la creación del vector `asignacion` y el vector `capacidad_residual`.

4. Compilación y ejecución

4.1. Entorno de ejecución

4.1.1. Hardware

- Procesador: Intel Core i7-6500U @ 2.50 GHz (4 núcleos).
- Memoria RAM: 8 GB.
- Almacenamiento: SSD de 256 GB.

4.1.2. Software

- Sistema Operativo: Ubuntu 22.04.4 LTS.
- Compilador: GCC 11.2.0 con estándar C++17.
- Bibliotecas utilizadas en C++: `vector`, `random`, `fstream` y `iostream`.

4.2. Instrucciones para compilación y ejecución

El proyecto incluye un archivo `Makefile` que automatiza el proceso de compilación. Para compilar el programa se debe ejecutar `make clean && make`. Esto genera el ejecutable `gap_simulator`, y el programa se ejecuta mediante `./gap_simulator`.

Al iniciarse, el ejecutable presenta por terminal un menú interactivo que permite:

- Seleccionar una instancia de prueba.
- Ejecutar las heurísticas constructivas.
- Ejecutar operadores de búsqueda local a partir de una solución inicial.
- Ejecutar la metaheurística GRASP indicando el valor de k .
- Cambiar de instancia durante la misma ejecución.
- Exportar las soluciones obtenidas a archivos de texto dentro del directorio `respuestas/`.

5. Experimentación

5.1. Experimento 1: ¿Qué heurística es mejor?

Hipótesis inicial: la segunda heurística es mejor que la primera porque no solo utiliza el costo de cada vendedor a un depósito como criterio de asignación, sino que además incorpora la cantidad de depósitos factibles a los que cada vendedor puede ser asignado mediante una variable de puntaje. Esto permite priorizar aquellos vendedores con menor cantidad de asignaciones posibles, reduciendo el riesgo de que luego no sean asignados y por ende penalizados.

Para evaluar esta hipótesis, se comparó el costo total promedio y la cantidad promedio de vendedores que no fueron asignados en ambas heurísticas sobre el conjunto de instancias de benchmark provistas. Los resultados se presentan en los siguientes gráficos:

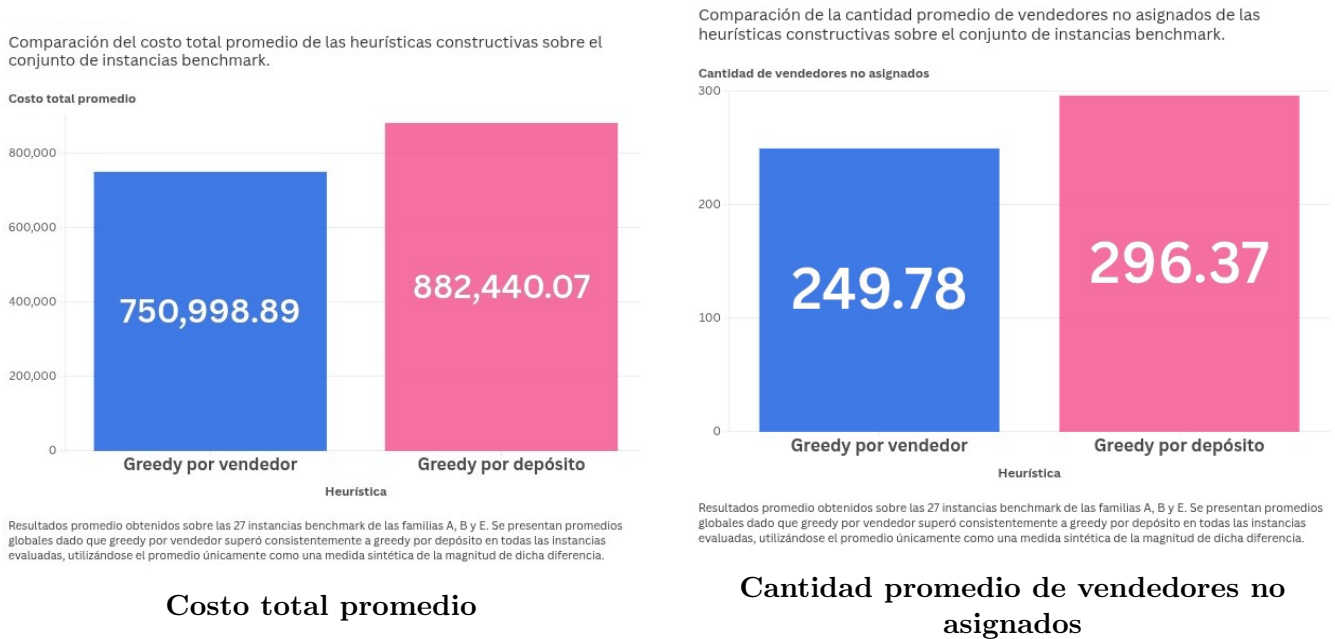


Figura 1: Comparación entre Greedy por vendedores y Greedy por depósito y puntaje.

A partir de los resultados se observa que, en términos de minimizar el costo total, la primera heurística devuelve una solución un 14,89 % mejor que la segunda. En cuanto a factibilidad de asignación, la primera heurística logra asignar un 15,72 % más de vendedores que la segunda.

Esto indica que la incorporación de la variable de puntaje en la segunda heurística, basada en la cantidad de depósitos factibles por vendedor, no mejora la calidad global de la solución; por el contrario, introduce decisiones que afectan negativamente tanto el costo total como la cantidad de vendedores asignados.

Para interpretar este comportamiento, se analizó la distribución de la cantidad de depósitos factibles por vendedor en las instancias benchmark, observándose una variabilidad nula dentro de cada instancia. Esto implica que dicho término aporta muy poca información discriminante al proceso de decisión, lo que se ve agravado por el hecho de que se calcula una única vez al inicio y no se actualiza dinámicamente durante la construcción de la solución. No se implementó su recálculo tras cada asignación por el alto costo computacional que esto implicaría.

Con el objetivo de evaluar la segunda heurística en escenarios donde la flexibilidad de los vendedores fuera más relevante, se generaron instancias modificadas con mayor variabilidad en la cantidad de depósitos factibles por vendedor. Sin embargo, este procedimiento también altera la competencia entre vendedores: al haber vendedores con muchas más alternativas que otros, disminuyen los conflictos por depósitos y el problema se vuelve globalmente más fácil. Por lo tanto, la mejora de costos observada en estas instancias

para ambas heurísticas refleja una menor dificultad intrínseca del escenario y no una mayor capacidad de la segunda heurística para explotar esa información. Por este motivo, dichas instancias fueron descartadas del análisis principal, ya que contaminarían la comparación entre heurísticas.

Se concluye entonces que, bajo el conjunto de instancias benchmark, la primera heurística, basada exclusivamente en costos, resulta más eficiente que la segunda, cuya variable de puntaje no logra aportar una mejora real a la calidad de la solución.

5.2. Experimento 2: ¿Swapear y relocalizar mejoran las soluciones obtenidas?

Hipótesis inicial: la búsqueda local para ambos operadores, swap y relocate, disminuye el costo total y la cantidad de vendedores no asignados de la solución.

Para evaluar esta hipótesis, se comparó el costo total promedio y la cantidad promedio de vendedores no asignados para ambos operadores de búsqueda local, aplicados sobre las soluciones generadas por las dos heurísticas, sobre el conjunto de instancias de benchmark provistas. Los resultados se presentan en las siguientes tablas:

Método	Costo promedio	No asignados promedio
H1 (heurística por vendedor)	750.999	249
H2 (heurística por depósito)	882.440	296
H1 + Swap	743.817	249
H2 + Swap	882.342	296
H1 + Relocate	750.999	249
H2 + Relocate	882.336	296

Cuadro 1: Costo promedio y cantidad de vendedores no asignados por método.

Método base	Mejora con Swap	Mejora con Relocate
Heurística por vendedor	0,96 %	0,00 %
Heurística por depósito	11 %	12 %

Cuadro 2: Mejora porcentual en costo promedio al aplicar búsqueda local.

A partir de los resultados se observa que el porcentaje de mejora alcanzado por ambos operadores depende fuertemente de la heurística de partida. Como H1 ya produce soluciones de mayor calidad, el margen de mejora es pequeño (0,96 % para Swap, 0 % para Relocate). H2, en cambio, parte de soluciones más débiles y ofrece mayor espacio de optimización, donde ambos operadores logran mejoras de alrededor del 11–12 %.

En cuanto a la cantidad de vendedores no asignados, esta permanece invariante en todos los casos: ambas heurísticas saturan las capacidades de los depósitos durante la construcción, y los movimientos locales simples no logran liberar suficiente capacidad para incorporar nuevos vendedores. Reducir este valor requeriría operadores más globales capaces de reconstruir parcialmente la solución.

Vale destacar que Relocate no logra ninguna mejora sobre H1 pero sí sobre H2. Esto se explica porque H1 genera soluciones con capacidades más homogéneamente distribuidas, donde mover vendedores individualmente raramente es beneficioso; H2, en cambio, llena depósitos de forma secuencial y puede dejar remanentes aprovechables por Relocate en los últimos depósitos iterados.

En conclusión, la hipótesis se verifica solo parcialmente: ambos operadores mejoran el costo total, especialmente sobre soluciones de menor calidad, pero ninguno logra reducir la cantidad de vendedores no asignados.

5.3. Experimento 3

Recordemos que la heurística por vendedor asigna cada vendedor al depósito factible de menor costo disponible, por lo que toma decisiones completamente determinísticas y puede quedar atrapada en óptimos locales. Se propone entonces incorporar aleatoriedad controlada mediante un greedy randomizado, en el que para cada vendedor se consideran los k mejores depósitos factibles y se selecciona aleatoriamente uno de ellos. La idea es permitir explorar soluciones distintas, evitando que decisiones tempranas potencialmente desfavorables condicionen toda la construcción de la solución.

Implementaremos nuestra metaheurística que sigue el esquema GRASP. En cada iteración se construye una solución mediante el greedy randomizado utilizando un valor fijo de k , y posteriormente se aplican los operadores de búsqueda local swap y relocate con el fin de mejorarla. Finalmente, se conserva la mejor solución encontrada entre todas las iteraciones realizadas. El número máximo de iteraciones se definió como $\min(500, 10 \times n)$, donde n es la cantidad de vendedores de la instancia, buscando aumentar la exploración en instancias de mayor tamaño sin que el tiempo de ejecución crezca excesivamente.

Hipótesis inicial: se espera que valores intermedios de k produzcan soluciones de mejor calidad que los extremos, dado que un valor pequeño de k genera un comportamiento muy cercano a la heurística por vendedor (poca exploración), mientras que uno demasiado grande introduce aleatoriedad excesiva que puede deteriorar la calidad promedio de las soluciones.

Para cada instancia y cada valor de k se realizaron 10 ejecuciones independientes, reportando valores promedio de costo total y cantidad de vendedores no asignados para reducir el efecto de fluctuaciones aleatorias. Los resultados de la metaheurística GRASP con distintos valores de k se presentan a continuación:



Figura 2: Resultados de GRASP con $k \in \{\max(2, m/4), m/2, m\}$.

A partir de los resultados se observa que el comportamiento esperado no se verifica: contrariamente a la hipótesis, los valores más grandes de k producen sistemáticamente soluciones de mejor calidad, tanto en costo total como en cantidad de vendedores no asignados.

Con el objetivo de entender este comportamiento, se comparó el desempeño de la heurística por vendedores contra el greedy randomizado ejecutado de forma aislada con $k = m$, es decir, permitiendo elegir aleatoriamente cualquier depósito factible.

Los resultados muestran que el greedy randomizado obtiene soluciones significativamente mejores que la heurística por vendedores incluso antes de aplicar búsqueda local o la metaheurística: en promedio, reduce el costo total en un 72,88 % y la cantidad de vendedores no asignados en un 83,35 %.

Este resultado sugiere que la principal ventaja de introducir aleatoriedad no proviene de escapar de óptimos locales, sino de una mejor distribución de la capacidad disponible entre depósitos. En las instancias benchmark, prácticamente todos los vendedores pueden asignarse a cualquier depósito, por lo que el problema no radica en encontrar depósitos factibles sino en administrar sus capacidades limitadas. La heurística por vendedores, al asignar siempre el depósito de menor costo disponible, provoca que muchos vendedores compitan por los mismos depósitos atractivos, saturándolos tempranamente y dejando a vendedores posteriores sin opciones viables. El greedy randomizado, al aceptar depósitos algo más costosos desde etapas tempranas, distribuye la carga de forma más homogénea y reduce considerablemente la cantidad de vendedores penalizados. El ahorro por evitar esas penalizaciones supera ampliamente el incremento de costo por asignaciones subóptimas puntuales.

Adicionalmente, se observa que la búsqueda local aporta solo mejoras marginales sobre las soluciones de la metaheurística, actuando principalmente como un refinamiento fino: la mayor parte de la ganancia proviene de la fase constructiva aleatorizada.

Se concluye que la aleatoriedad controlada es una estrategia muy efectiva para estas instancias, y que valores grandes de k son los más adecuados, contradiciendo la hipótesis inicial. El beneficio no está en escapar de óptimos locales sino en una distribución más equilibrada de las capacidades desde la fase de construcción.

6. Análisis del caso real

La instancia real consiste en 1100 vendedores y 310 depósitos. Antes de evaluar los métodos, se analizaron las características estructurales de la instancia y se las comparó con las instancias de benchmark para entender en qué medida las conclusiones de los experimentos anteriores son transferibles.

La observación estructural más visible es la cantidad de depósitos factibles por vendedor antes de realizar cualquier asignación. En los benchmarks, como se vio en el Experimento 1, todos los vendedores de una misma instancia inicialmente tienen exactamente la misma cantidad de opciones factibles, igual al número de depósitos. En el caso real, esto también ocurre ya que cada vendedor puede ser asignado a uno de entre los 310 depósitos. Esto confirma que la heurística por depósito tampoco aporta valor en la instancia real ya que la variable de flexibilidad no tiene poder discriminante cuando todos los vendedores tienen exactamente las mismas alternativas. Esto sugiere que la restricción operativa en el caso real no está en la factibilidad de asignación individual sino en la administración de la capacidad residual de cada depósito.

Respecto a la factibilidad global, la capacidad total de la red (15.235 unidades) supera la demanda total (10.707 unidades) con un excedente del 42 %. Si bien, a diferencia de la instancia real, las instancias benchmark no poseen una demanda total única, ya que para cada vendedor el consumo de capacidad depende del depósito elegido, experimentalmente se observó que suelen presentar una cantidad significativa de vendedores no asignados, en contraste con el caso real que fue siempre nula. Esta diferencia explica por sí sola el comportamiento invertido del greedy randomizado: en los benchmarks, la aleatoriedad era beneficiosa porque distribuía la carga y evitaba penalizaciones costosas por vendedores no asignados. En el caso real, donde la capacidad sobra y todos los métodos logran asignar a la totalidad de los vendedores, introducir aleatoriedad solo incrementa el costo de asignación sin ninguna ganancia en factibilidad. Por eso el greedy randomizado empeora sistemáticamente a medida que aumenta k , invirtiendo el patrón del Experimento 3.

Método	Costo	No asignados
H1	788	0
H2	1770	0
Greedy randomizado ($k = 77$)	5867	0
Greedy randomizado ($k = 155$)	8853	0
Greedy randomizado ($k = 310$)	10514	0
H1 + Swap	730	0
H2 + Swap	1251	0
H1 + Relocate	788	0
H2 + Relocate	757	0
GRASP $k = 77$	742	0
GRASP $k = 155$	762	0
GRASP $k = 310$	767	0

Cuadro 3: Resultados de cada método sobre la instancia real.

Los resultados sobre la instancia real se presentan en el Cuadro 3. H1 + Swap obtiene el menor costo, seguido de GRASP con $k=77$ ejecutado con una sola iteración. La configuración original de 500 iteraciones no logró finalizar, mientras que una única iteración demandó 39.999 ms. Esto se debe a que cada iteración ejecuta un greedy randomizado seguido de una búsqueda local swap cuyo número de mejoras no está acotado, haciendo que GRASP sea inviable en tiempo real para esta escala. Esto responde negativamente la tercera pregunta de ThunderPack. En cambio, H1 + Swap constituye la recomendación concreta: obtiene la mejor solución, asigna a todos los vendedores y ejecuta en tiempo razonable. Sin embargo, a partir de los resultados sobre el benchmark, no puede garantizarse el mismo comportamiento ante escenarios con capacidad más limitada. En conclusión, la red de ThunderPack posee capacidad suficiente para absorber la demanda y es posible obtener asignaciones de buena calidad en tiempo acotado, respondiendo afirmativamente las dos primeras preguntas planteadas por la empresa.

7. IA utilizada:

Estos son los links a nuestras conversaciones con LLMs que contienen las ayudas provistas para la resolución del Trabajo:

- Clases: [Conversación con Gemini](#)
- Experimentos: [Conversación con ChatGPT](#) y [Conversación con Claude](#)

8. Conclusión final

El trabajo abordó la asignación de vendedores a depósitos para ThunderPack mediante heurísticas, búsqueda local y GRASP. La calidad relativa de cada método depende de cuán ajustada está la capacidad a la demanda: con capacidad restringida, la aleatoriedad en la construcción distribuye mejor la carga y reduce penalizaciones; con capacidad amplia, como en el caso real, esa aleatoriedad deja de aportar y los métodos deterministas con búsqueda local resultan preferibles. No existe entonces una estrategia única óptima, sino que el método debe ajustarse a la relación capacidad-demanda. Para ThunderPack se recomienda entonces priorizar heurísticas deterministas con swap por su balance entre calidad y tiempo asumiendo que sus instancias tienen una amplia capacidad disponible.